

Session Recap — Mockito Learning Project (Order Service) — Stage 1

Workflow

1. Defined the project scope: one evolving Spring Boot service (`OrderService`), taught through 5 stages, rather than separate throwaway projects.
2. Built the skeleton: `Order`, `OrderStatus`, `PaymentClient` (interface), `OrderRepository` (interface), `PaymentFailedException`, `OrderService.placeOrder()`.
3. Wrote the first Mockito test: happy-path — payment succeeds → order saved as `CONFIRMED`.
4. Covered dependencies/interfaces conceptually in depth before touching more code.

Problems / Resolutions

Question raised	Resolution
Why use an interface (<code>PaymentClient</code>) instead of a concrete class?	Interfaces let Mockito generate clean fakes with no real logic to fight against, and decouple <code>OrderService</code> from any one vendor implementation (Dependency Inversion).
Isn't writing both a real implementation AND a mock contradictory?	No — a real implementation, Mockito's fake, and even a lambda are all just different <i>fulfillments</i> of the same contract. <code>OrderService</code> never knows which one it's holding.
Can you create a bare <code>new PaymentClient()</code> ?	No — Java forbids instantiating interfaces directly; there's no method body to run. A class or generated proxy must always supply the behavior.
Where does <code>paymentClient</code> 's actual object come from inside <code>OrderService</code> ?	Never created inside <code>OrderService</code> — it's <i>injected</i> via the constructor (Dependency Injection), either by Spring (production) or Mockito (tests).
Why hardcode <code>o.setId(1L)</code> in the <code>thenAnswer</code> stub?	Simulating what a real database's auto-increment would do on <code>save()</code> , since there's no real database in a unit test.
Why <code>getArgument(0)</code> specifically?	Zero-indexed position of the parameter in the method being stubbed — <code>save(Order order)</code> has one parameter, at index 0.
Is <code>@InjectMocks</code> used because we're only testing business logic?	Confirmed — <code>@InjectMocks</code> goes on the class containing the logic under test; <code>@Mock</code> goes on its dependencies/collaborators.

Q&A Covered

- Difference between testing a service (mock the repository) vs. testing a repository itself (`@DataJpaTest`, real database, no mocking).
- Full list of JUnit / AssertJ / Mockito methods relevant across all 5 stages, prioritized by when they're needed.
- Realistic expectation-setting: memorizing the API isn't the goal — pattern recognition plus IDE autocomplete is how it works day to day in real projects.

Mockito / JUnit / AssertJ Concepts Mastered So Far

- `@Mock`
- `@InjectMocks`
- `@ExtendWith(MockitoExtension.class)`
- `when(...).thenReturn(...)`
- `when(...).thenAnswer(...)`
- `ArgumentMatchers.any()`
- `invocation.getArgument(n)`
- `assertThat(...).isEqualTo(...)`
- `assertThat(...).isNotNull()`

Next Up — Stage 2 (in progress)

- `thenThrow(...)` — simulating a failure from a dependency
- `assertThatThrownBy(...)` — asserting the code throws the right exception
- `verify(mock, never()).method(...)` — proving a method was never called
- `ArgumentCaptor<T>` — capturing the actual object passed to a mock

Full Project Reference (as of Stage 1)

`PaymentClient.java`

```
java

package com.example.orderservice.client;

import java.math.BigDecimal;

public interface PaymentClient {
    boolean charge(String customerId, BigDecimal amount);
}
```

OrderRepository.java

java

```
package com.example.orderservice.repository;

import com.example.orderservice.model.Order;
import org.springframework.data.jpa.repository.JpaRepository;

public interface OrderRepository extends JpaRepository<Order, Long> {
}
```

OrderStatus.java

java

```
package com.example.orderservice.model;

public enum OrderStatus {
    PENDING,
    CONFIRMED,
    PAYMENT_FAILED
}
```

Order.java

java

```

package com.example.orderservice.model;

import jakarta.persistence.*;
import java.math.BigDecimal;

@Entity
@Table(name = "orders")
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String customerId;
    private BigDecimal amount;

    @Enumerated(EnumType.STRING)
    private OrderStatus status;

    public Order() {}

    public Order(String customerId, BigDecimal amount, OrderStatus status) {
        this.customerId = customerId;
        this.amount = amount;
        this.status = status;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getCustomerId() { return customerId; }
    public void setCustomerId(String customerId) { this.customerId = customerId; }
    public BigDecimal getAmount() { return amount; }
    public void setAmount(BigDecimal amount) { this.amount = amount; }
    public OrderStatus getStatus() { return status; }
    public void setStatus(OrderStatus status) { this.status = status; }
}

```

PaymentFailedException.java

```
java
```

```
package com.example.orderservice.exception;

public class PaymentFailedException extends RuntimeException {
    public PaymentFailedException(String message) {
        super(message);
    }
}
```

OrderService.java

java

```
package com.example.orderservice.service;

import com.example.orderservice.client.PaymentClient;
import com.example.orderservice.exception.PaymentFailedException;
import com.example.orderservice.model.Order;
import com.example.orderservice.model.OrderStatus;
import com.example.orderservice.repository.OrderRepository;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;

import java.math.BigDecimal;

@Service
public class OrderService {

    private static final Logger log = LoggerFactory.getLogger(OrderService.class);

    private final OrderRepository orderRepository;
    private final PaymentClient paymentClient;

    public OrderService(OrderRepository orderRepository, PaymentClient paymentClient) {
        this.orderRepository = orderRepository;
        this.paymentClient = paymentClient;
    }

    public Order placeOrder(String customerId, BigDecimal amount) {
        if (amount == null || amount.signum() <= 0) {
            throw new IllegalArgumentException("Order amount must be positive");
        }

        boolean paymentSuccess = paymentClient.charge(customerId, amount);

        if (!paymentSuccess) {
            log.warn("Payment failed for customer {} amount {}", customerId, amount);
            throw new PaymentFailedException("Payment declined for customer " + customerId);
        }

        Order order = new Order(customerId, amount, OrderStatus.CONFIRMED);
        Order saved = orderRepository.save(order);
        log.info("Order {} confirmed for customer {}", saved.getId(), customerId);
    }
}
```

```
return saved;
```

```
}
```

```
}
```

OrderServiceTest.java (Stage 1)

java

```
package com.example.orderservice.service;

import com.example.orderservice.client.PaymentClient;
import com.example.orderservice.model.Order;
import com.example.orderservice.model.OrderStatus;
import com.example.orderservice.repository.OrderRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.math.BigDecimal;

import static org.assertj.core.api.Assertions.assertThat;
import static org.mockito.Mockito.when;

@ExtendWith(MockitoExtension.class)
class OrderServiceTest {

    @Mock
    private OrderRepository orderRepository;

    @Mock
    private PaymentClient paymentClient;

    @InjectMocks
    private OrderService orderService;

    @Test
    void placeOrder_whenPaymentSucceeds_savesConfirmedOrder() {
        // ARRANGE
        String customerId = "cust-1";
        BigDecimal amount = new BigDecimal("500.00");

        when(paymentClient.charge(customerId, amount)).thenReturn(true);

        when(orderRepository.save(org.mockito.ArgumentMatchers.any(Order.class)))
            .thenAnswer(invocation -> {
                Order o = invocation.getArgument(0);
                o.setId(1L);
            });
    }
}
```

```
        return o;  
    });  
  
    // ACT  
    Order result = orderService.placeOrder(customerId, amount);  
  
    // ASSERT  
    assertThat(result).isNotNull();  
    assertThat(result.getId()).isEqualTo(1L);  
    assertThat(result.getStatus()).isEqualTo(OrderStatus.CONFIRMED);  
    assertThat(result.getCustomerId()).isEqualTo(customerId);  
}  
}
```